Expert Systems With Applications 285 (2025) 127827

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

SCRIPT: A Scalable Continual Reinforcement Learning Framework for Autonomous Penetration Testing

Shicheng Zhou [a,b], Jingju Liu [a,b,c,*], Yuliang Lu [a,b,**], Jiahai Yang [c], Yue Zhang [d], Bo Lin [d], Xiaofeng Zhong [a,b], Shulong Hu [a]

[a] College of Electronic Engineering, National University of Defense Technology, Hefei, 230037, China
[b] Anhui Province Key Laboratory of Cyberspace Security Situation Awareness and Evaluation, Hefei, 230037, China
[c] Institute for Network Sciences and Cyberspace and the Beijing National Research Center for Information Science and Technology, Tsinghua University, Beijing, 10084, China
[d] College of Computer Science and Technology, National University of Defense Technology, Changsha, 410073, China

ARTICLE INFO

Keywords:
Cybersecurity
Penetration testing
Reinforcement learning
Continual reinforcement learning
Catastrophic forgetting
Policy transfer

ABSTRACT

Compared with manual-based penetration testing (pentesting), autonomous pentesting offers advantages in autonomy and efficiency, emerging as a trending research area. Reinforcement learning (RL) is a natural fit for studying this topic. However, the proliferation of interconnected devices and systems has expanded attack surfaces, coupled with the constant emergence of new vulnerabilities, making pentesting tasks and scenarios in the real world non-stationary. This non-stationarity presents a new challenge for RL-based autonomous pentesting, as it requires pentesting agents to possess strong generalization capabilities, thereby achieving continuous learn new tasks while mitigating catastrophic forgetting. Previous research in this field overlooked this real-world challenge. To this end, we present the first scalable continual RL framework for autonomous pentesting, namely SCRIPT. SCRIPT trains the agent to learn continually like humans in a large-scale task sequence, allowing it to realize positive forward transfer (leveraging previously learned knowledge to accelerate new task learning) and resist catastrophic forgetting. Inspired by humans' lifelong learning process, we separate SCRIPT's learning process into new task learning and knowledge consolidation processes. Specifically, we introduce a scaffolded curriculum learning approach to bootstrap new task learning through rolling exploration and policy imitation. Besides, we propose a policy retention method that utilizes regularization-based weight importance constraints and progressive self-distillation-based policy retrospection to mitigate catastrophic forgetting. Experiments are conducted using Vulhub, a widely used repository of vulnerable Docker environments. Results demonstrate that SCRIPT effectively enables agents to achieve positive forward transfer and enhance their resistance to catastrophic forgetting, making it a feasible approach for pentesting agents to handle the real-world non-stationarity challenge and tackle expanded attack surfaces.

1. Introduction

Penetration testing, shortly pentesting or PT, is an effective methodology for conducting offensive cybersecurity assessments. Its primary objective is to analyze the attack surfaces and pinpoint potential vulnerabilities within operating systems or applications from an attacker's perspective via executing controlled attacks. The increasing numbers of interconnected devices and systems provide cybercriminals with an ever-expanding array of potential vulnerabilities to exploit, thereby expanding the attack surfaces. This poses a challenge for manual-based pentesting, as it heavily relies on skilled professionals with domain-specific knowledge, which is costly, time-consuming, and personnel-constrained (Holm, 2023). In contrast, autonomous pentesting offers advantages in autonomy and efficiency, emerging as a trending research area. Reinforcement learning (RL) is a natural fit for studying autonomous pentesting. On the one hand, pentesting can be modeled as a dynamic sequential decision-making problem, while RL stands as a well-established method for optimizing such decisions. On the other hand, RL provides a learning paradigm of training agents to learn policies through trial and error while interacting with the environment

* Corresponding author at: College of Electronic Engineering, National University of Defense Technology, Hefei, 230037, China.
** Corresponding author at: College of Electronic Engineering, National University of Defense Technology, Hefei, 230037, China.
E-mail addresses: zhoushicheng@nudt.edu.cn (S. Zhou), liujingju17@nudt.edu.cn (J. Liu), luyuliang@nudt.edu.cn (Y. Lu), yang@cernet.edu.cn (J. Yang), zhangyue@nudt.edu.cn (Y. Zhang), linbo19@nudt.edu.cn (B. Lin), zhongxiaofeng17@nudt.edu.cn (X. Zhong), hsl98@nudt.edu.cn (S. Hu).

https://doi.org/10.1016/j.eswa.2025.127827
Received 13 February 2025; Received in revised form 13 April 2025; Accepted 19 April 2025
Available online 8 May 2025
0957-4174/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

without supervision or environmental models. In the context of RL-based autonomous pentesting, a task typically refers to the process where agents autonomously gather hosts' information and exploit vulnerabilities within specific vulnerable scenarios. Different scenarios define different tasks, requiring agents to learn distinct attack policies accordingly.

Pentesting tasks or scenarios in real-world settings are *non-stationary*. This non-stationarity arises because the attack surfaces constantly expand as new systems, applications, and vulnerabilities emerge, requiring agents to continuously learn new tasks. However, previous research on RL-based autonomous pentesting (e.g., Chen et al. (2023), Ilic et al. (2024), Li et al. (2023), Schwartz and Kurniawati (2019)) has mainly focused on training agents within fixed scenarios. Although a few works (Li et al., 2024; Yang et al., 2023; Zhou, Liu, Lu, Yang, Hou, et al., 2024) have attempted to enable agents to quickly adapt to dynamic environments, this adaptation is typically achieved through retraining, and after adapting to the new environment, agents often forget the knowledge learned before. As such, these agents often exhibit poor generalization, making it difficult for them to handle non-stationarity scenarios (e.g., newly emerged vulnerability types), thereby struggling to tackle expanding attack surfaces. We attribute this limitation to *catastrophic forgetting* (Zhou & Cao, 2021), where learning one new task results in the loss of previously acquired knowledge, and agents fail to leverage past knowledge to accelerate adaptation to new tasks.

We argue that generalizable and effective pentesting agents should be able to learn continually like humans, which means they can resist catastrophic forgetting and leverage previously learned knowledge to rapidly adapt to new tasks or scenarios. This ability is commonly referred to as continual learning or incremental learning. Continual learning provides a foundation for continual reinforcement learning (CRL) (Abel et al., 2023; Khetarpal et al., 2022; Ring, 1995), empowering RL agents to maintain previously learned skills while rapidly acquiring new ones (Wang et al., 2023) in non-stationary environments. Such features are highly desirable for agents in real-world applications, thus motivating us to propose a feasible CRL solution for autonomous pentesting.

Considering the diversity of vulnerability environments, when applying CRL to training pentesting agents, two key challenges need to be addressed: the stability–plasticity trade-off (Wang et al., 2024), and task scalability. The stability–plasticity trade-off requires agents to strike a balance between robust policy stability (to preserve past knowledge and resist catastrophic forgetting) and adaptive policy plasticity (to rapidly learn new tasks via positive policy transfer). Scalability refers to agents' capability to be trained on a large number of sequential tasks without clear task boundaries and access to previous scenarios.

Existing CRL methods can be conceptually divided into three categories: replay-based, expansion-based, and regularization-based (Wang et al., 2023). However, they struggle to address the above challenges well enough to adapt to pentesting tasks. Reasons are as follows: **(a)** Replay-based CRL methods typically depend on large memory capacities to store past experience samples (Riemer et al., 2019; Rolnick et al., 2019) or pseudo-samples generated by an additional generative model (Atkinson et al., 2021; Gao et al., 2021), while the memory capacity requirements of these approaches limit their task scalability and practical applications in real-world scenarios; **(b)** Expansion-based methods either require the incremental expansion of the model architecture (Hung et al., 2019; Rusu,Rabinowitz, et al., 2016; Schwarz et al., 2018; Yoon et al., 2018) or rely on explicit task boundaries and hand-designed heuristics to integrate new knowledge (Gaya et al., 2023; Wang et al., 2023), which also results in limited scalability and increased computational complexity. In contrast, regularization-based methods offer advantages in task scalability, as they require no model architecture growth, no storing of previous experience samples, and no explicit task boundaries. However, as tasks increase, some regularization-based methods (Kaplanis et al., 2019; Lan et al., 2023; Zhou & Cao, 2021) typically overlook the need for plasticity and leave significant room for improvement in addressing catastrophic forgetting.

To this end, we present a **S**calable **C**ontinual **R**e**I**nforcement learning framework for autonomous **P**enetration **T**esting, namely SCRIPT.[1] To the best of our knowledge, this is the first feasible framework for training pentesting agents to learn in a continual fashion. Our framework draws inspiration from the human lifelong learning process (Parisi et al., 2019): when facing numerous learning tasks, human beings can accelerate new task learning by leveraging accumulated experiential knowledge; afterward, they can consolidate the learned knowledge to avoid forgetting. Thus, to realize scalable continual learning and balance the stability–plasticity trade-off, we separate the agent's learning process into new task learning and knowledge consolidation processes, which ensures that old knowledge is not disturbed when learning new tasks.

Specifically, **during the new task learning process,** we use two key methods: first, inspired by scaffolding instruction in education (Van de Pol et al., 2010), we introduce a scaffolded curriculum learning approach to accelerate new task learning by leveraging prior knowledge. Additionally, model re-initialization is applied to prevent catastrophic loss of plasticity (Dohare et al., 2021). Then, **during the knowledge consolidation process**, we combine the regularization-based CRL approach with knowledge distillation to mitigate catastrophic forgetting. The use of knowledge distillation is reflected in two aspects: first, it achieves policy transfer through policy distillation (Czarnecki et al., 2019); second, we propose a method for progressively retrospecting previously learned knowledge through self-distillation (Kim et al., 2020; Wang & Yoon, 2022), which we term progressive self-distillation-based policy retrospection. Finally, we conduct comparison experiments and ablation studies to answer the following research questions (RQs):

- **RQ1**: Does SCRIPT have better resistance compared to other methods against catastrophic forgetting?
- **RQ2**: How do the two methods used in SCRIPT, weights importance constraint and policy retrospection, contribute to agents' resilience against catastrophic forgetting?
- **RQ3**: Can SCRIPT achieve positive forward transfer during the learning process?
- **RQ4**: How do the scaffolded curriculum guide and model re-initialization affect the forward transferability of SCRIPT?

The main contributions of our work are threefold:

- We introduce SCRIPT, the first scalable continual reinforcement learning framework for autonomous pentesting, enabling agents to leverage previously learned knowledge to accelerate new task learning while avoiding catastrophic forgetting.
- Inspired by the scaffolding instruction, we propose a scaffolded curriculum learning approach for realizing positive forward transfer, which can effectively utilize a prior policy to accelerate the new task learning through rolling exploration and policy imitation. Besides, we propose a policy retention method involving regularization-based weight importance constraints and progressive self-distillation-based policy retrospection to retain previously learned knowledge.
- We continuously train agents to perform pentesting tasks in 40 high-fidelity vulnerability environments built using Vulhub. The experimental results show that SCRIPT can memorize over 60% previously learned tasks and leverage previously learned knowledge to rapidly adapt to new tasks. This demonstrates that SCRIPT is task-scalable, achieves positive forward transfer, and improves resistance to catastrophic forgetting.

[1] https://github.com/Joe-zsc/Script

2. Related work

Our work belongs to the CRL application for the autonomous pentesting field, so we mainly discuss the representative-related works around the two fields.

2.1. Autonomous penetration testing

Research on autonomous pentesting originated from automated attack planning, which aimed to use planning algorithms to discover all possible attack paths in target systems. Classical planning approaches in this domain included attack trees and attack graphs (Schneier, 1999; Sheyner et al., 2002), which offered interpretable and formal models for evaluating network security. However, both approaches assumed complete knowledge of the target network topology and hosts' configurations, which was unrealistic for real-world pentesting experts.

Recently, RL has shown remarkable success in various domains, including video games (Vinyals et al., 2019), autonomous vehicles (Feng et al., 2023), and notably, cybersecurity (Hore et al., 2023). Compared to traditional planning methods, RL provides an agent–environment interaction learning paradigm. It focuses on training an agent to learn a policy through trial-and-error interaction with the environment, eliminating the need for supervision or environmental models. These attributes position RL as a suitable approach for studying autonomous pentesting.

Previous research on RL-based autonomous pentesting mainly focused on finding attack paths in highly abstract simulations like NASim (Schwartz & Kurniawatti, 2019) or CyberBattleSim (Microsoft Defender Research Team, 2021). They made efforts to improve the exploration efficiency of the pentesting agents through various methods such as action space decomposition (Li et al., 2023; Tran et al., 2021), action space constraints (Yang et al., 2025), or the introduction of expert demonstration data (Chen et al., 2023), thus realizing efficient attack path decision-making in unrealistic scenarios with abstract pentesting actions. Yang et al. (2023) and Li et al. (2024) also proposed solutions for finding attack paths in dynamic simulated network scenarios, while their methods still suffer from catastrophic forgetting. Besides, finding attack paths in existing abstract simulations relies on the over-optimistic and unrealistic assumption that agents can observe the target hosts' configurations or network information in advance from a global perspective, which is unrealistic from real-world attackers' perspective (Zhou, Liu, Lu, Yang, Hou, et al., 2024).

There also have been some research attempts at applying RL methods in practical pentesting tools (Maeda & Mimura, 2021; Takaesu, 2018), or constructing training environments using virtual machines (Nguyen et al., 2024). However, these efforts trained agents to learn policies in single and fixed scenarios. Consequently, such learned policies frequently suffer from poor generalization, as they tend to overfit specific training scenarios or tasks, hindering their adaptability to the diverse and non-stationary network environments encountered in the real world. This limitation poses challenges to the widespread deployment of pentesting agents.

Compared with previous research, we believe that pentesting agents with strong generalization ability and high applicability must be able to continuously learn and adapt to new tasks in non-stationary environments, so as to address the expanding attack surfaces encountered in the real world. To this end, we for the first time present a feasible CRL solution for training autonomous pentesting agents that can realize positive forward transfer and mitigate catastrophic forgetting. Besides, following the settings of Zhou, Liu, Lu, Yang, Hou, et al. (2024), we conduct experiments in simulated scenarios constructed based on virtualized vulnerable environments, which offer controlled, isolated, and high-fidelity training scenarios.

2.2. Continual reinforcement learning

The suitable CRL method for autonomous pentesting should address the following problems: (a) showing positive forward transfer by leveraging past experience; (b) mitigating catastrophic forgetting; (c) scalable to a large number of tasks; (d) without clear task boundaries.

Recently, numerous CRL approaches have been investigated to address the above challenges, which can be conceptually separated into three categories: replay-based, expansion-based, and regularization-based (Wang et al., 2023).

Replay-based approaches require a substantial replay buffer to store past experience samples (Riemer et al., 2019; Rolnick et al., 2019), or they can utilize a generative model to create pseudo-samples (Atkinson et al., 2021; Gao et al., 2021) from previous tasks. These stored samples can then be reused for rehearsal or interleaved with online updates while learning a new task. However, as the number of tasks grows, the memory capacity demands of these approaches hinder their scalability and practical utilization in real-world scenarios.

Similarly, expansion-based approaches face challenges in terms of limited scalability and computational complexity, as they incrementally expand the model architecture (Hung et al., 2019; Rusu,Rabinowitz, et al., 2016; Schwarz et al., 2018; Yoon et al., 2018) to safeguard past weights from being perturbed by new information. Additionally, some of these approaches heavily rely on explicit task boundaries or hand-designed heuristics for incorporating new knowledge (Gaya et al., 2023; Wang et al., 2023).

By contrast, regularization-based approaches (Kaplanis et al., 2019; Lan et al., 2023; Schwarz et al., 2018) retain old knowledge by incorporating regularization terms that impose constraints on network weight updates, thereby avoiding substantial changes in important weights. These approaches are easy to implement and exhibit improved scalability, but they still confront the challenge of balancing stability and plasticity as the number of tasks increases.

Compared with existing research, this paper introduces a novel scalable CRL framework that combines regularization-based methods and curriculum learning, aiming to achieve improved performance retention and positive forward transferability in a large-scale task sequence. Our approach also draws on the core principles of replay-based and expansion-based techniques by taking advantage of their complementary strengths while minimizing their weaknesses. Unlike replay-based methods, our approach does not necessitate access to or storage of past data or tasks. Instead, we learn from previous task knowledge via self-distillation. Additionally, in contrast to expansion-based techniques, our method maintains a consistent number of parameters, eliminating the need for architecture growth, clear task boundaries, or task-specific parameters. But inspired by the progressive network architecture (Rusu,Rabinowitz, et al., 2016; Schwarz et al., 2018), we separate the new task learning and knowledge consolidation models to achieve a stability–plasticity trade-off and task scalability.

3. Problem formulation

RL is a machine learning method that maps the state of the environment to actions, aiming to train an agent to learn the optimal policy with maximum cumulative reward through interaction with the environment. This kind of agent–environment interaction paradigm makes RL a natural fit for studying continual learning (Khetarpal et al., 2022) and autonomous pentesting. In this section, we first formulate the problem statement of RL and CRL. Then, we introduce how we model pentesting as a Markov Decision Process to apply RL and CRL for autonomous pentesting.

3.1. Reinforcement learning

A typical RL problem can be modeled as a finite, discrete-time Markov Decision Process (MDP) that is formalized by a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{P}, \gamma)$, where $\mathcal{S}$ represents the state space, $\mathcal{A}$ is the action space, $\mathcal{R} : \mathcal{S} \times \mathcal{A} \mapsto \mathbb{R}$ is the reward function, $\mathcal{P} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \mapsto [0, 1]$ is the state transition probability function and γ is the discount factor used to determine the importance of long-term rewards (François-Lavet et al., 2018; Sutton & Barto, 2018).

Formally, for a standard RL framework, at each time step t the RL agent observes the state s_t from the environment and selects an action a_t based on its policy π. Then, the agent receives a reward signal from the environment, and the environment transitions to the next state s_{t+1}. The goal of the RL agent is to find the optimal policy π^* with internal parameter θ^* that maximizes the expected cumulative reward (Wang et al., 2023). Following Zhu et al. (2023) and Zhang et al. (2023), we define the objective function as

$$J(\theta) = \mathbb{E}_{\tau \sim D_{\pi_\theta}} \sum_{i=0}^{T} \gamma^t R_t, \tag{1}$$

where $\gamma \in [0, 1]$ is the discount factor used to determine the importance of long-term rewards, τ is a trajectory $(s_0, a_0, s_1, a_1, \ldots, s_T)$, D_{π_θ} denotes the distribution of τ under policy π_θ.

3.2. Continual reinforcement learning

For RL in a changing environment, following Wang et al. (2022) and Zhang et al. (2023), we consider the dynamic environment as a sequence of stationary tasks $\mathcal{T} = [T_1, \ldots, T_{i-1}, T_i, \ldots]$ on a certain timescale, where each task corresponds to specific environment characteristics during the associated time period. The time period is assumed to be long enough for the agent to gather sufficient experience samples to complete policy learning for the associated task. Suppose there is a space of MDPs $\mathcal{M} = [M_1, \ldots, M_{i-1}, M_i, \ldots]$, where each $M_i \in \mathcal{M}$ models a specific task that remains stationary within its time period. The environment of the task changes over time, resulting in a non-stationary environment distribution, and the agent remains unknown to the current environment's identity (Wang et al., 2022).

In this setting, the goal of the CRL agent is to learn a single policy that maximizes rewards on all learned tasks $[T_1, T_2, \ldots, T_n]$ (Zhang et al., 2023), with learning parameters optimized as

$$\theta^* = \arg\max_{\theta} \sum_{i=1}^{n} J_{M_i}(\theta), \tag{2}$$

where J_{M_i} denotes the agent's learning objective of the i_{th} task. The agent is supposed to learn a sequence of tasks one by one to retain previously learned knowledge while learning new tasks. In our research, the overall performance of the learned policy across all tasks serves as the primary metric for evaluating the performance of CRL agents.

Considering the practical requirements of pentesting tasks and the task scalability needs, following Schwarz et al. (2018), we assume that during the continual learning process, the agent cannot revisit or re-access tasks it has previously learned, meanwhile there are no clear boundaries between tasks. These assumptions, on the one hand, allow the agent to handle the non-stationarity challenge of real-world pentesting without being constrained by pre-defined task boundaries, and, on the other hand, align with the practical pentesting scenario, where the agent typically faces new target systems and does not have the luxury of revisiting past tasks.

3.3. Model penetration testing as MDP

The pentesting task can be viewed as a dynamic sequential decision-making process that involves executing controlled attacks on target hosts based on gathered information, followed by the detection of potential vulnerabilities. As a powerful machine learning paradigm for optimal sequential decision-making, RL is a natural fit for studying autonomous pentesting.

Note that in our research, we aim to train an applicable pentesting agent with its policy that can be generalized to various scenarios. Therefore, we focus on the fundamental and essential pentesting decision task of training agents to learn to compromise different target hosts, enabling it to learn and adapt to diverse scenarios continually, provided the training scenarios are rich enough. This idea differs from previous works that concentrated on training RL agents to find optimal attack paths within specific simulated networks. Following Zhou, Liu, Lu, Yang, Hou, et al. (2024), we refrain from incorporating network topology as a factor in both agents modeling and the training scenarios. This decision stems from the understanding that, at the network level, both target host selection and attack path decisions are predominantly mission-oriented, rendering this strategy non-generalizable even for pentesting experts. In our setting, the attack path decision can be made as a separate study, e.g., heuristic search, or generic path planning strategies such as depth-first search or breadth-first search (Zhou, Liu, Lu, Yang, Hou, et al., 2024).

We model the pentesting task as an MDP, where the interaction between the agent and environment is depicted in Fig. 1. At its core, the interaction process is an Observe, Orient, Decide, Act (OODA) loop (Brehmer, 2005):

- Observe. The agent observes the environment (vulnerable target hosts) via scanning tools and then receives the raw state information, which can be used for further decision-making. The content of the raw state information corresponds to the state space $\mathcal{S}$ in the MDP, which refers to the configuration of target hosts covering opened ports, running services, operating systems, etc.
- Orient. Encode the raw state information using the embedding technique so that it can be converted into a state vector s_t that can be received by the neural network model. In our research, we pre-trained a Sentence-BERT model (Wang et al., 2021) as the state information encoder, as the obtained raw state information is usually in text form. Embedded representations of state vectors can (a) capture key features of raw state information in a latent space; (b) realize end-to-end learning; and (c) avoid the dimensional explosion of state space.
- Decide. The agent (an RL model) receives the state vector as an input and outputs an action a_t from the action space $\mathcal{A}$. In our research, the action space covers a wide range of Tactics, Techniques, and Procedures (TTPs), including information gathering (e.g., port scanning, service scanning, operating system detection, etc.) and vulnerability exploitation (all exploits in Metasploit Framework (MSF)[2]).
- Act. The agent calls and executes the appropriate tools like KALI Linux[3] or Metasploit Framework via the application programming interface.

Besides, the agent needs to update its policy based on environmental feedback, which is given by the reward function $\mathcal{R}$. The agent's objective is to gather additional information about the target host to improve the inference of potential vulnerabilities, exploit these vulnerabilities to compromise the host and minimize the overall cost of actions to maintain covert operations. To this end, following Zhou, Liu, Lu, Yang, Hou, et al. (2024), the reward function $\mathcal{R}$ is defined as:

$$\mathcal{R} = \text{Value}(h) - \textstyle\sum_{a \in A} \text{Cost}(a), \tag{3}$$

where $A \subseteq \mathcal{A}$ is the set of actions used in the pentesting process, $\text{Value}(h)$ returns a positive reward value if the target host h is compromised by exploiting the correct vulnerability, or the agent successfully gains some kind of information about the host. $\text{Cost}(a)$ refers to the cost of action a. $\mathcal{T}$ remains unknown as we use model-free RL algorithms to train the agent.

[2] https://github.com/rapid7/metasploit-framework
[3] https://www.kali.org/

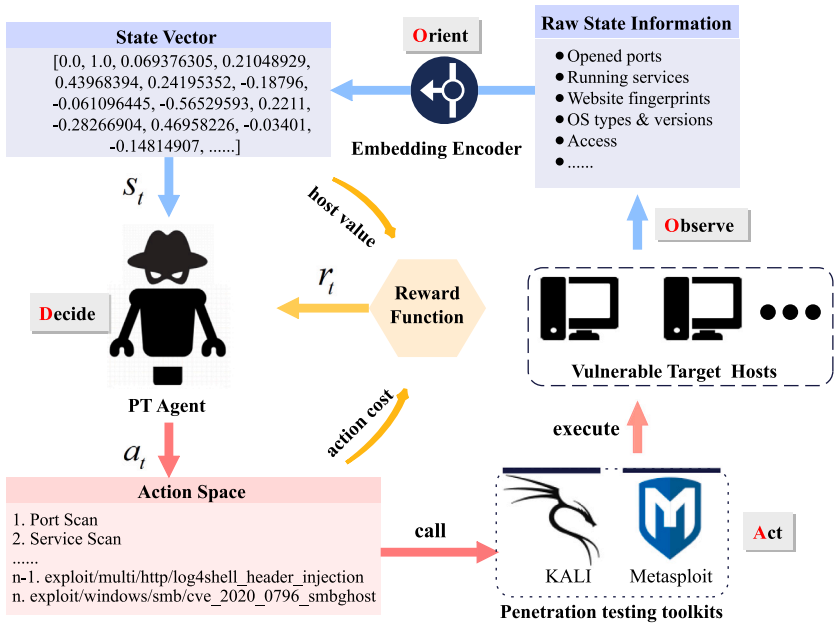

Fig. 1. The interaction process between the PT agent and environments.

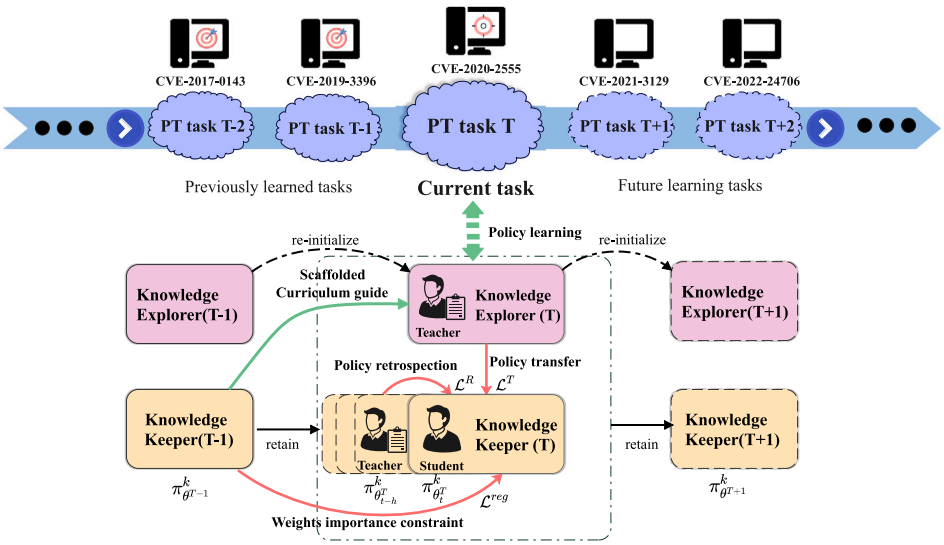

Fig. 2. Overview of SCRIPT. SCRIPT is designed to continuously train the pentesting agent to learn attack policies within task sequences. Each target host refers to a task, as each host has different configurations as well as vulnerabilities, so the agent needs to learn different policies to compromise them. The framework follows a two-phase process that iterates alternately between new task learning (green lines) and knowledge consolidation (red lines). In the **new task learning phase**, we initialize a knowledge explorer for standard RL training. During this phase, the knowledge keeper with previous tasks' knowledge provides a scaffolded curriculum guide to accelerate new task learning. In the **knowledge consolidation phase**, the knowledge explorer transfers its newly learned knowledge to the knowledge keeper, meanwhile, the knowledge keeper preserves the previously learned knowledge from being forgotten.

4. Proposed method

In this section, we introduce our proposed framework SCRIPT combined with Proximal Policy Optimization (PPO) algorithm (Schulman et al., 2017), and how we train the RL agent to learn pentesting tasks in a continual fashion. It is noteworthy that SCRIPT is not limited to PPO. A wide range of actor–critic paradigm RL algorithms can be used out of the box. The full workflow of SCRIPT is illustrated in Fig. 2, which we will elaborate on in the subsequent sections.

4.1. Framework overview

The core concept of SCRIPT lies in the notion that a human student can perpetually absorb knowledge from adept teachers across different subjects and subsequently consolidate this diverse knowledge after class by reviewing. Inspired by this, we separate the continual learning process into new task learning and knowledge consolidation processes, allowing the agent to achieve accelerated learning while avoiding forgetting, thereby realizing continual learning on a large scale of tasks.

In SCRIPT, teachers possessing specialized task knowledge are designated as knowledge explorers, while students striving to acquire proficiency in various tasks are termed knowledge keepers. The knowledge explorer operates as a full actor–critic architecture for standard RL training, whereas the knowledge keeper can be seen as a simple actor-network as it does not follow the policy updating paradigm of RL. The two components are learned in alternating phases (new task learning and knowledge consolidation), which are illustrated in Fig. 2.

It is important to note that the roles of teacher and student shift at different stages of learning. During the new task learning phase, we aim to train the knowledge explorer to learn task-specific knowledge quickly. In this phase, the knowledge keeper assumes the role of a teacher, providing scaffolded curriculum guidance to bootstrap the knowledge explorer with their prior knowledge. In the phase of knowledge consolidation, our objective shifts to transferring the newly acquired knowledge to the knowledge keeper while ensuring the retention of past knowledge. Here, the knowledge explorer transitions into the role of a teacher, distilling task knowledge for the knowledge keeper. Additionally, we introduce a progressive self-distillation-based policy retrospection method and a weighted importance constraint approach to preserve the knowledge keeper's previously acquired knowledge. At this juncture, the keeper embodies both the teacher and student roles.

4.2. New task learning

We separate the new task learning model, allowing SCRIPT to realize positive forward transfer when a new task is introduced and thereby avoiding learning from scratch. The full procedure of the new task learning phase is described in Algorithm 1.

For the current task T, the knowledge explorer interacts with the task environment and is trained to learn the policy (denoted as $\pi^E_{\phi_T}$) using the PPO algorithm (Schulman et al., 2017). Meanwhile, the policy of the knowledge keeper with previously learned knowledge (denoted as $\pi^K_{\theta^{T-1}}$) remains fixed. For brevity, we denote π^E and π^K to represent their policy, respectively. Note that when learning a new task, the knowledge explorer is re-initialized to avoid catastrophic loss of plasticity (Dohare et al., 2021) (Line 1 in Algorithm 1). This stems from the understanding that RL systems tend to overfit to experiences collected early in training, leading to failure in adapting to a changing world (Abbas et al., 2023). This phenomenon is evident in experiments conducted in Section 5, particularly shown in Fig. 8.

Before describing our method, we have to give a brief introduction to the PPO algorithm. PPO is an on-policy actor–critic method with two primary variants of PPO: PPO-Penalty and PPO-Clip, where we focus on the most widely used variant PPO-Clip (Schulman et al., 2017). By using PPO, the knowledge explorer updates policy π^E_ϕ by taking multiple steps of (usually minibatch) SGD to optimize the following objective:

$$\mathcal{L}^{PPO}(\phi) = \mathbb{E}_{\tau_t \sim \pi}\left[\min\left(r_t(\phi)\hat{A}_t, \text{clip}\left(r_t(\phi), 1-\epsilon, 1+\epsilon\right)\hat{A}_t\right)\right], \tag{4}$$

where $r_t(\phi) = \frac{\pi_\phi(a_t|s_t)}{\pi_{\phi_{\text{old}}}(a_t|s_t)}$ denote the probability ratio, $\hat{A}_t$ is an estimator of the advantage function which describes how much better the action is than others on average, $\text{clip}\left(r_t(\phi), 1-\epsilon, 1+\epsilon\right)$ modifies the surrogate objective by clipping the probability ratio, and ϵ is a small hyperparameter which roughly measures how far away the new policy is allowed to go from the old.

In the absence of other aids, the knowledge explorer needs to improve its policy via trial and error. The most challenging aspect of this process occurs at the outset when a policy, devoid of any prior experiences, must randomly encounter rewards to facilitate further improvement. This inefficiency is evident. In contrast, when humans encounter a difficult new task, they typically benefit from the curriculum guidance of a teacher. Scaffolding instruction (O'Rourke et al., 2015) is an educational approach that provides structured and gradual guidance to learners as they acquire new skills or knowledge. This approach ensures that learners can successfully complete tasks they might not be able to do independently at first. Inspired by this concept, we propose a scaffolded curriculum learning approach to bootstrap new task learning by leveraging a prior or guiding policy at the early learning stage. The proposed scaffolded curriculum learning can effectively shorten the exploration and learning process for common knowledge in the general pentesting process, such as the scanning and probing actions during the early stages of pentesting. This improvement can be reflected in agents' jump-start performance (see Fig. 9).

Within our framework, the knowledge keeper assumes the role of a suitable prior policy, given its theoretical understanding of past tasks. Serving as the prior policy, it offers initial guidance in gathering experiences with non-zero rewards and provides a role model for the knowledge explorer to mimic, despite not being fully optimal on its own for the current task. We remark that the method of using pre-existing policy to bootstrap RL algorithms was similar to Uchendu et al. (2023). However, they mainly focused on improving exploration for value-based RL methods. In contrast, we extend this concept to include policy-based methods, and more importantly, we apply a policy distillation approach to effectively alleviate the influence of distributional shifts, thereby accelerating learning.

Specifically, we divide the new task learning stage into two processes: curriculum learning, which leverages the "not bad" guiding policy to transition the knowledge explorer into "good" states, followed by independent learning to acquire specific task knowledge using a standard RL learning paradigm.

Next, we focus on the scaffolded curriculum learning stage and delve into two methods: rolling exploration and policy imitation, both of which are utilized to effectively bootstrap new task learning by leveraging a guiding policy.

4.2.1. Rolling exploration

Scaffolding instruction provides support to learners in a structured and gradual manner as they learn new tasks. Following this concept, we employ the fixed π^K as the guiding policy and engage in roll-in with both π^K and π^E to enhance exploration.

Assuming the horizon of each training episode is denoted as H, we define a control policy $\tilde{\pi}$ that takes the current state s_t as input, outputs the action $a_t \sim \tilde{\pi}(\cdot|s_t)$ to interact with the environment, and receives the reward signal r_t (Lines 6 and 7 in Algorithm 1). Subsequently, the trajectory (s_t, a_t, r_t, s_{t+1}) is stored in a replay buffer for optimizing π^E. Before a particular guiding length $l < H$, we deploy π^K as the control policy for l steps, followed by π^E taking over control of action output for the remaining $H - l$ steps until the horizon H is reached. This process can be symbolically described as $\tilde{\pi}_{1:l} = \pi^K, \tilde{\pi}_{l+1:H} = \pi^E$. During the initial training phases, π^K outperforms the untrained π^E. l is initialized to a large value so that we can gather experiences using π^K to gather more experiences with positive rewards. Then we gradually shift the control policy away from π^K and toward π^E, linearly reducing $l \to 0$ over the training episodes (Line 4 in Algorithm 1). This is known as the *progressive scaffolding instructional strategy*. This process resembles a multi-stage curriculum designed to progressively reduce the influence of the guiding policy π^K throughout training, facilitating on-policy experience for the learned policy.

The experiences provided by the teacher π^K are often out of distribution (OOD) for the learner π^E. Once the teacher's guidance is removed, the distributional shift can cause learners to struggle with completing tasks independently. This phenomenon is known as the *scaffolding fading effect*, which occurs because learners rely too heavily on the teacher's guidance without internalizing it into their own knowledge. This phenomenon is observable in experiments in Section 6.2.2, notably in Fig. 10.

4.2.2. Policy imitation

To avoid the scaffolding fading effect and help the student internalize the teacher's guidance into their own knowledge, we also attempt to have the knowledge explorer mimic the knowledge keeper by policy distillation, thus it can learn more robust and generalizable policy, eliminating the effect of distributional shift during policy optimization (Igl et al., 2021).

Policy distillation is a method used to transfer knowledge from a teacher policy to a student policy without damaging interference (Rusu, Colmenarejo, et al., 2016). This process in this case involves the student (knowledge explorer) policy π^E imitating the guiding (knowledge

keeper) policy π^K by minimizing the divergence of action distributions between them, which is denoted as

$$\mathcal{L}^D(\pi^E_\phi) = \mathbb{E}_{\tau_t \sim \tilde{\pi}}\left[D_{KL}\left(\pi^E_{\phi^T}(\cdot|s_t), \pi^K_{\theta^{T-1}}(\cdot|s_t)\right)\right], \tag{5}$$

where D_{KL} refers to the KL-Divergence loss function, and $\tau_t \sim \tilde{\pi}$ means the trajectories come from a mixed sampling of the two policies.

Overall, in the curriculum learning stage, the knowledge explorer updates its policy π_ϕ following a linear combination of PPO loss and distillation loss (Line 14 in Algorithm 1):

$$\mathcal{L}(\pi^E_\phi) = \mathcal{L}^{PPO} + \alpha\mathcal{L}^D, \tag{6}$$

where α represents a coefficient that measures the extent of imitation toward the guiding policy. We suppose that the coefficient should be correlated with the deviation between the new and old policies. During policy iteration, if the deviation between the new policy and the old policy (the guiding policy) is significant, then policy imitation should be intensified; conversely, it should be attenuated. It is worth noting that in PPO, the probability ratio coincidentally assesses the deviation between the old and new policies, prompting us to set $\alpha_t = |1 - r_t(\phi)|$ in implementations.

After the curriculum learning stage, the knowledge explorer achieves kickstarting. Subsequently, it steps into the independent learning stage, where it fully takes control of the action policy, explores the specific task knowledge, and updates its policy following the common PPO algorithm without the guidance of the knowledge keeper.

Algorithm 1 SCRIPT – New Task Learning

Input: Task T, policy of the knowledge explorer $\pi^E_{\phi^T}$, policy of the knowledge keeper $\pi^K_{\theta^{T-1}}$ with previously learned knowledge, horizon of each training episode H, guiding length $l < H$, replay buffer $\mathcal{B}$ with size N

Output: $\pi^E_{\phi^T}$

1: Randomly initialize $\pi^E_{\phi^T}$
2: **for** each training episode **do**
3: Reset the training environment to the initial state $s_0 \sim env_T$
4: linearly reducing $l \to 0$ over the training episodes
5: **for** each timestep t **do**
6: $a_t \sim \tilde{\pi}(\cdot|s_t)$, where $\tilde{\pi}_{1:l} = \pi^K_{\theta^{T-1}}, \tilde{\pi}_{l+1:H} = \pi^E_{\phi^T}$ ▷ Rolling exploration
7: Execute action a_t, receives reward signal r_t, observes next state s_{t+1}
8: Store the transition $\tau_t = (s_t, a_t, r_t, s_{t+1})$ in replay buffer $\mathcal{B} \leftarrow \mathcal{B} \cup \{\tau_t\}$
9: $s_t \leftarrow s_{t+1}$
/* Policy optimization */
10: **if** $t \mod N = 0$ **then**
11: **for** each policy update iteration **do**
12: Sample minibatch of transitions $(s_i, a_i, r_i, s_{i+1}) \sim \mathcal{B}$
13: **if** $l > 0$ **then**
14: Update $\pi^E_{\phi^T}$ following Eq. (6) ▷ Policy imitation
15: **else**
16: Update $\pi^E_{\phi^T}$ with PPO method following Eq. (4)

4.3. Knowledge consolidation

During the knowledge consolidation phase, we aim to carefully consolidate the knowledge explorer's newly learned task knowledge into the knowledge keeper. In other words, the knowledge keeper is supposed to learn a policy $\pi^K_{\theta^T} = \left\{\pi^E_{\phi^T}, \pi^K_{\theta^{T-1}}\right\}$ in conjunction with task-specific policy and its previously learned policy. At this phase, the parameters of the knowledge explorer's policy ϕ^T are frozen. For brevity, we use π^K_θ and π^E_ϕ to represent $\pi^K_{\theta^T}$ and $\pi^E_{\phi^T}$, respectively. The full procedure of the knowledge consolidation phase is described in Algorithm 2.

The consolidation process entails two key components: policy transfer and policy retention. Policy transfer involves transferring task knowledge from the knowledge explorer to the knowledge keeper. Meanwhile, policy retention ensures that previous task knowledge is retained and not forgotten during the transfer process. Specifically, we introduce a policy retention method that combines two techniques: regularization-based weight importance constraints and progressive self-distillation-based policy retrospection. Both aim to prevent the model's parameters from deviating significantly from their previously learned values, thereby alleviating the issue of catastrophic forgetting. Policy transfer and retention work simultaneously to update the knowledge keeper's policy, and we refer to these training iterations as consolidation iterations.

Note that the proposed policy retention method essentially belongs to the regularization-based CRL approach. Some representative baseline methods, including Fine-tune, Progress&Compress (Schwarz et al., 2018), Policy Consolidation (Kaplanis et al., 2019), and MeDQN (Lan et al., 2023), are compared in Section 6.1.1 to demonstrate the superiority of our method.

4.3.1. Policy transfer

We apply policy distillation to transfer the newly learned policy $\pi^E_{\phi^T}$ from the knowledge explorer to the knowledge keeper, while the distinction in this part is that the knowledge explorer assumes the role of the teacher.

Before distillation, we use π^E_ϕ to generate a trajectory buffer on task T (Line 1 in Algorithm 2): $\mathcal{B}^E_T = \left\{(s_i, a_i, r_i, s_{i+1})\right\}_{i=0}^N$, and then minimize the KL-divergence between the action distributions of π^K_θ and π^E_ϕ as follows:

$$\mathcal{L}^T(\pi^K_{\theta_t}) = \mathbb{E}_{s\sim\mathcal{B}^E_T}\left[D_{KL}\left(\pi^K_{\theta_t}(\cdot|s), \pi^E_\phi(\cdot|s)\right)\right], \tag{7}$$

where θ_t represents the parameters of the knowledge keeper's policy at consolidation iteration t (Line 4 in Algorithm 2).

4.3.2. Weights importance constraint

In order to guard against catastrophic forgetting during consolidation iterations, we apply a weight importance constraint method, the online Elastic Weight Consolidation (online EWC) (Schwarz et al., 2018), to our framework. Similar regularization-based constraint methods like Memory Aware Synapses (MAS) (Aljundi et al., 2018) and Intelligent Synapses (IS) (Zenke et al., 2017) can serve as alternative options within our framework.

EWC (Kirkpatrick et al., 2016) presents an approximate Bayesian solution to mitigate catastrophic forgetting. Its core insight lies in sequentially incorporating task-related information into the posterior, thereby avoiding catastrophic forgetting, as the resulting posterior is independent of task ordering. Online EWC, a variant of EWC, enhances adaptability by dynamically updating its regularization term during training. This flexibility enables Online EWC to scale effectively to a large number of tasks.

Particularly, during consolidation iterations the Online EWC (Schwarz et al., 2018) regularization term with respect to the parameters θ_t is calculated as

$$\mathcal{L}^{reg}(\pi^K_{\theta_t}) = \frac{1}{2}\left\|\pi^K_{\theta_t} - \pi^K_{\theta^{T-1}}\right\|^2_{\gamma F^*_{T-1}}, \tag{8}$$

where γ in this equation is a hyperparameter that determines the importance of previous tasks relative to the current task (Line 6 in Algorithm 2). A higher value of γ prioritizes the preservation of knowledge from previous tasks, resulting in slower adaptation to the current task and potentially better retention of past knowledge. F^*_{T-1} is the mean and diagonal Fisher of the online EWC Gaussian approximation resulting from previous tasks. The Fisher information matrix captures the curvature of the parameter space and provides an estimate of the sensitivity of the loss function to changes in the model parameters.

4.3.3. Policy retrospection

Humans can consolidate previously learned knowledge and prevent forgetting by reviewing. In machine learning, self-distillation is proposed to utilize knowledge from itself, without the involvement of an extra teacher model (Wang & Yoon, 2022). Inspired by this, we propose a policy retrospection method based on self-distillation to analogize the process of review in the human learning process.

To achieve this goal, we utilize the historical information as a virtual teacher to train the knowledge keeper to mimic its own predictions, thus facilitating a form of policy retrospection. Specifically, we consistently save the policy parameters of the knowledge keeper (represented as $\pi^K_{\theta_{t-h}}$) from a preceding iteration $t-h$, where h denotes the retrospect horizon (Line 9 in Algorithm 2). During the consolidation iterations, the knowledge keeper progressively distills its previously acquired knowledge into itself for further refinement (Line 5 in Algorithm 2):

$$\mathcal{L}^R(\pi^K_{\theta_t}) = \mathbb{E}_{s\sim\mathcal{B}^E_T}\left[\mathcal{D}_{KL}\left(\pi^K_{\theta_t}(\cdot|s), \pi^K_{\theta_{t-h}}(\cdot|s)\right)\right]. \tag{9}$$

By employing policy retrospection, the knowledge keeper learns to smooth out its predictions and reduces the likelihood of extreme parameter updates that could lead to significant deviations from previously learned values. Essentially, self-distillation acts as a regularization technique that generalizes better and is less prone to overfitting, encouraging the model to maintain consistency with its past knowledge while adapting to new data or tasks while adapting to new tasks.

Overall, the knowledge keeper realizes the transfer of new task knowledge and the retention of old task knowledge through policy transfer and policy retention. At each consolidation iteration, the knowledge keeper updates its policy following a linear combination of regularization terms:

$$\mathcal{L}(\pi^K_\theta) = \mathcal{L}^T + \lambda\mathcal{L}^{reg} + \mu\mathcal{L}^R, \tag{10}$$

where λ and μ are regularization strengths (Line 7 in Algorithm 2).

Algorithm 2 SCRIPT – Knowledge Consolidation

Input: Task T, policy of the knowledge explorer $\pi^E_{\phi^T}$, policy of the knowledge keeper $\pi^K_{\theta^{T-1}}$, replay buffer $\mathcal{B}$ with size N, retrospect horizon h

Output: $\pi^K_{\theta^T}$

1: Collect N transitions $(s_i, a_i, r_i, s_{i+1}) \sim \mathcal{B}$ from environment of T using $\pi^E_{\phi^T}$
2: **for** each consolidation iteration t **do**
3: Sample minibatch of transitions $(s_i, a_i, r_i, s_{i+1}) \sim \mathcal{B}$
4: Calculate the policy transfer loss $\mathcal{L}^T(\pi^K_{\theta_t})$ following Eq. (7)
5: Calculate the policy retrospection loss $\mathcal{L}^R(\pi^K_{\theta_t})$ following Eq. (9)
6: Calculate the regularization loss $\mathcal{L}^{reg}(\pi^K_{\theta_t})$ following Eq. (8)
7: Update policy by minimizing $\mathcal{L}(\pi^K_\theta)$ in Eq. (10)
8: **if** $t \mod h = 0$ **then**
9: Save $\pi^K_{\theta^T_t}$ as the old policy for policy retrospection

5. Experimental settings

We conducted all experiments on a 64-bit server equipped with an Intel (R) Xeon(R) Gold 6354 CPU @ 3.00 GHz, 256 GB memory, and an 80 GB NVIDIA A800 Ampere GPU, running Ubuntu 22.04. PyTorch serves as the backend for the implementation of the framework. Our code, scenarios, data, and all hyperparameters are publicly available for further replicability and future research: https://github.com/Joe-zsc/Script.

5.1. Penetration testing scenario

Following Zhou, Liu, Lu, Yang, Hou, et al. (2024), Zhou, Liu, Lu, Yang, Zhang and Chen (2024), we construct high-fidelity pentesting scenarios using Vulhub,[4] an open-source repository of vulnerable Docker environments. Training agents by interacting directly with virtualization platforms is feasible but can be sample-inefficient and time-consuming due to transmission delay and action execution. Thus, to balance the sampling efficiency and environmental fidelity during training, we construct simulated training environments based on Vulhub. Specifically, we pre-probe all vulnerability-related information in the agent's state space (such as open ports, services, operating systems, website fingerprints, etc.) using information scanning tools within the agent's action space, and store these details in JSON format. These simulated environments are constructed based on actual feedback data gathered from real vulnerable hosts. Therefore, theoretically, the agent interacting with these simulated environments is equivalent to interacting with real vulnerable hosts.

Totally, we have created 45 vulnerability environments (see Table 1) to evaluate agents' performance. These vulnerability environments (denoted as V) target various representative vulnerable products. And more importantly, reliable and stable exploits of these vulnerabilities are available in MSF for the agent to use. We see these vulnerability environments as a sequence of tasks, each of them having a different host configuration and vulnerability, thereby representing a distinct training scenario. The pentesting agent is expected to learn how to compromise them continually without suffering forgetting. All of the training scenarios, as well as the scenario-building method, are available in our code repository.

5.2. Agent setting

During the training process, the agent begins in the initial state, lacking any prior knowledge or access to target hosts. Its tasks are to continually compromise the target hosts one by one via information-probing and vulnerability exploitation within limited steps.

We represent the raw state information (in text form) into state vectors by finetuning the pre-trained embedding model Sentence-BERT (Wang et al., 2021) using our dataset gathered from exploit modules and vulnerability descriptions in the MSF and NVD[5] repository. It is worth noting that existing pre-trained BERT models can also serve as state information encoders. In our approach, we specifically finetune the pre-trained models using a pentesting-related corpus to ensure the state vector effectively captures the semantic meaning of the scan information.

Rewards provide crucial numerical feedback to the agent regarding its actions. This feedback guides the agent's learning progress toward achieving its goals. In accordance with Eq. (3) and following Zhou, Liu, Lu, Yang, Hou, et al. (2024), our reward function is established using subjective estimates provided by pentesting experts. Specifically, a reward of +1000 is granted if the agent effectively exploits the correct vulnerability and compromises the target host. Additionally, in our experiments, the agent is encouraged to obtain more information about the target host to reason more accurately about the possible vulnerabilities of the target. Thus, we assign a reward of +100 for successfully gathering information about the target (i.e., the OS type and version, running services, website fingerprint, etc.). Conversely, opting for incorrect or illogical actions incurs a penalty of −10.

Agents' available actions in training scenarios cover TTPs. Totally, in our experiments the size of action space is 2278, covering 2274 vulnerability exploits[6] and 4 information probing actions.

[4] https://github.com/vulhub/vulhub
[5] https://nvd.nist.gov
[6] We use the Metasploit Framework version v6.2.31-dev in implementations.

Table 1
Vulnerability environments.

CVE-ID	Vulnerability name
CVE-2016-4437	Apache Shiro Code Execution Vulnerability
CVE-2016-5734	PhpMyAdmin Remote Code Execution Vulnerability
CVE-2017-1000353	Jenkins Remote Code Execution Vulnerability
CVE-2017-10271	Oracle Corporation WebLogic Server Remote Code Execution Vulnerability
CVE-2017-11610	Supervisord Remote Code Execution Vulnerability
CVE-2017-12635	Apache CouchDB JSON Remote Privilege Escalation Vulnerability
CVE-2017-17562	Embedthis GoAhead Remote Code Execution Vulnerability
CVE-2017-5638	Apache Struts Remote Code Execution Vulnerability
CVE-2017-7494	Samba Remote Code Execution Vulnerability
CVE-2017-8291	Artifex Ghostscript Type Confusion Vulnerability
CVE-2017-8917	Joomla! SQL injection vulnerability
CVE-2018-1000861	Jenkins Stapler Web Framework Deserialization of Untrusted Data Vulnerability
CVE-2018-11776	Apache Struts Remote Code Execution Vulnerability
CVE-2018-12613	phpMyAdmin File Inclusion Vulnerability
CVE-2018-2628	Oracle WebLogic Server Unspecified Vulnerability
CVE-2018-7600	Drupal Core Remote Code Execution Vulnerability
CVE-2019-0230	Apache Struts Remote Code Execution Vulnerability
CVE-2019-11043	PHP FastCGI Process Manager Buffer Overflow Vulnerability
CVE-2019-15107	Webmin Command Injection Vulnerability
CVE-2019-17558	Apache Solr VelocityResponseWriter Plug-In Remote Code Execution Vulnerability
CVE-2019-3396	Atlassian Confluence Server and Data Center Server-Side Template Injection Vulnerability
CVE-2019-9193	PostgreSQL Remote Code Execution Vulnerability
CVE-2020-10199	Sonatype Nexus Repository Remote Code Execution Vulnerability
CVE-2020-13945	Apache APISIX Default Access Token Vulnerability
CVE-2020-16846	SaltStack Salt Shell Injection Vulnerability
CVE-2020-17530	Apache Struts Remote Code Execution Vulnerability
CVE-2020-2555	Oracle Multiple Products Remote Code Execution Vulnerability
CVE-2020-7961	Liferay Portal Deserialization of Untrusted Data Vulnerability
CVE-2020-9496	Apache OfBiz Remote Code Execution Vulnerability
CVE-2021-22205	GitLab Community and Enterprise Editions Remote Code Execution Vulnerability
CVE-2021-22986	F5 BIG-IP & BIG-IQ Centralized Management iControl REST Remote Code Execution Vulnerability
CVE-2021-25646	Apache Druid Remote Code Execution Vulnerability
CVE-2021-3129	Laravel Ignition File Upload Vulnerability
CVE-2021-32682	elFinder Archive Command Injection Vulnerability
CVE-2021-41773	Apache HTTP Server Path Traversal Vulnerability
CVE-2021-42013	Apache HTTP Server Path Traversal Vulnerability
CVE-2021-44228	Apache Log4j2 Remote Code Execution Vulnerability
CVE-2022-0543	Debian-specific Redis Server Lua Sandbox Escape Vulnerability
CVE-2022-22947	VMware Spring Cloud Gateway Code Injection Vulnerability
CVE-2022-24706	Apache CouchDB Insecure Default Initialization of Resource Vulnerability
CVE-2023-32315	Ignite Realtime Openfire Path Traversal Vulnerability
CVE-2023-21839	Oracle WebLogic Server Unspecified Vulnerability
CVE-2023-38646	Metabase Pre-Auth Remote Code Execution Vulnerability
CVE-2023-42793	JetBrains TeamCity Authentication Bypass Vulnerability
CVE-2023-46604	Apache ActiveMQ Deserialization of Untrusted Data Vulnerability

5.3. Implementation details

In our experiments, for each new task, we train the agent (the knowledge explorer) to learn over 500 episodes. Each episode has a maximum of 100 steps, allowing for early termination when the task is complete. We choose the optimal hyperparameters using a grid search approach for the best experimental performance. Specifically, for each new task learning phase, the first 50 episodes are the curriculum learning stage, where we use the knowledge keeper to bootstrap task learning and reduce the guiding length l linearly from 50 to 0. During the knowledge consolidation phase, the agent (the knowledge keeper) undergoes 1000 consolidation iterations, where the retrospect horizon h is set to 1000, and the regularization strengths λ and μ are set to 2000 and 1, respectively. The detailed list of the hyperparameter settings can be found in the open-sourced code repository.

5.4. Metrics

A scalable CRL method should achieve a stability–plasticity trade-off in a large-scale task sequence. Therefore, in each run, to evaluate agents' resilience against catastrophic forgetting and learning performance on each task, we randomly select 40 targets from Table 1 as the training task sequences $\mathcal{T} = [T_1, \ldots, T_{40}]$, $T_i \in V$. Besides, we also randomly choose one of the remaining targets ($T_m \notin \mathcal{T}$) to evaluate agents' learning efficiency on the unseen target. All the experiments are repeated for 5 runs with different random seeds and the average values are reported.

5.4.1. Resilience against catastrophic forgetting

We evaluate agents' resilience against catastrophic forgetting by assessing their performance retention on all previously learned tasks, as well as forgetfulness on the initial task during sequential training in $\mathcal{T}$.

Performance retention (PR) is defined as how much of all past tasks the agent can still accomplish after learning the current task T_n. In our implementation, we evaluate agents' zero-shot performance on all previously learned tasks, including the current one, sequentially after learning each task (see results in Figs. 3(a) and 5(a)). Thus, performance retention somewhat represents the agent's forgetting curve during the continual learning process. Mathematically, PR is defined as

$$PR(n) = \frac{\sum_{i=1}^{n} C(T_i)}{n}, \tag{11}$$

where $C(T_i) = 1$ indicates that the task T_i can be accomplished by the agent while $C(T_i) = 0$ signifies failure. $PR = 1$ means that the agent has memorized all the previously learned knowledge. Besides, we also record the total number of agents' memorized tasks after learning all tasks (see results in Fig. 4).

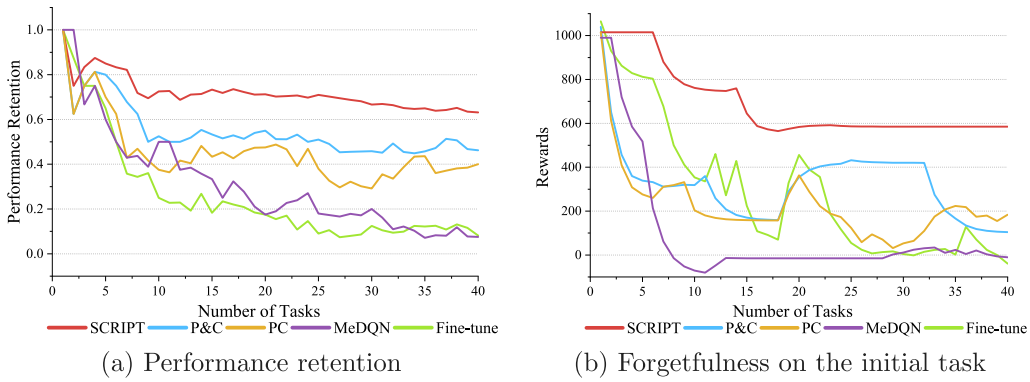

Fig. 3. Changes in performance retention (a) and zero-shot performance on the initial task (b) across different methods, reflecting the agents' forgetting curves and their forgetfulness on the initial task in the sequential learning process, respectively.

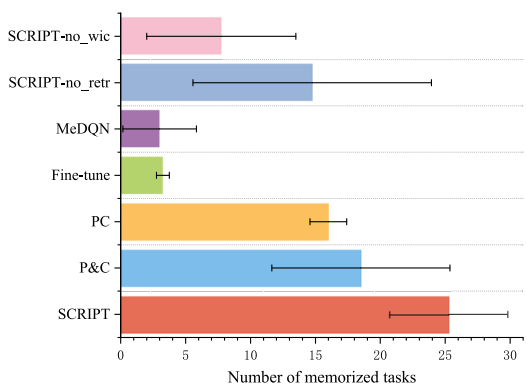

Fig. 4. Number of memorized tasks after learning 40 tasks. Error bars represent the standard deviation of the mean.

Additionally, inspired by Powers et al. (2022), we also evaluate agents' zero-shot performance on the initial task T_1 after they have learned each subsequent task $T_i \in \mathcal{T}$. The forgetfulness is reflected by the change in the cumulative reward obtained by agents (see results in Figs. 3(b) and 5(b)). Long-term memory is essential for preventing catastrophic forgetting because if an agent can effectively store information in long-term memory, it can preserve previously learned knowledge even while acquiring new tasks. Besides, we also evaluate agents' short-term and medium-term memory capabilities in the ablation study (Fig. 6) to provide a comprehensive assessment of how new task knowledge impacts agents' previously learned knowledge.

5.4.2. Forward transferability

Forward transferability refers to whether agents can take advantage of previously learned tasks to facilitate learning new tasks. In each run, we evaluate forward transferability in two ways: firstly, by assessing agents' learning performance on the same unseen task T_m after learning varying numbers of tasks (see results in Fig. 7), and secondly, by investigating the cumulative training steps used by agents to learn each task during sequential training in $\mathcal{T}$ (see results in Fig. 8). The agent's learning performance can be reflected in the convergence speed of its learning curve. A faster convergence speed indicates better learning performance. Therefore, as the number of learning tasks increases, the agent's ability to achieve faster convergence on the same unseen task can serve as a good measure of its ability to achieve positive forward transfer. The cumulative training steps used by the agent to learn each task reflect the total timesteps it interacts with the environment. Fewer cumulative training steps indicate that the agent can complete the training for the task more quickly. During the sequential training process, whether the cumulative training steps effectively decrease can also serve as a good measure of the agent's ability to achieve positive forward transfer.

Furthermore, to assess the contribution of the scaffolded curriculum guide to the agent's forward transferability, following Zhu et al. (2023), we also evaluate jump-start performance on each task in $\mathcal{T}$. Jump-start performance refers to the cumulative reward obtained by the agent in the first training episode. This measures how quickly the agent can leverage previously learned knowledge to achieve meaningful rewards in the initial training episode, thereby achieving kickstarting (see results in Fig. 9).

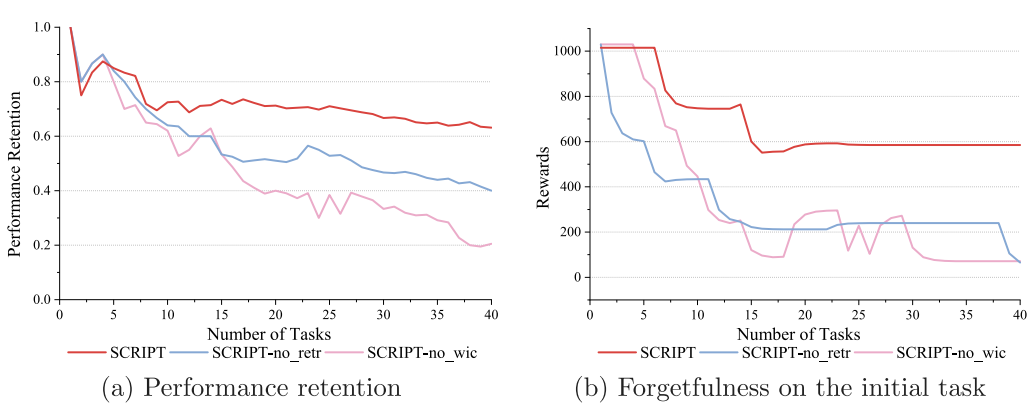

Fig. 5. Changes in performance retention (a) and zero-shot performance on the initial task (b) across SCRIPT and its variants during sequential training.

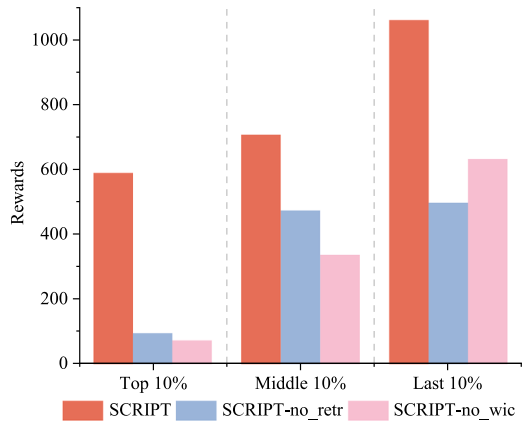

Fig. 6. Zero-shot performance of SCRIPT and its variants on the top 10%, middle 10%, and last 10% of the training tasks after all tasks are learned.

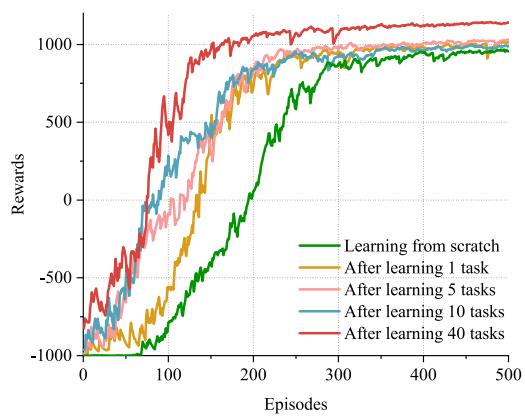

Fig. 7. Forward transferability of SCRIPT. Each line represents the agent's learning curve on an unseen task after learning a different number of tasks.

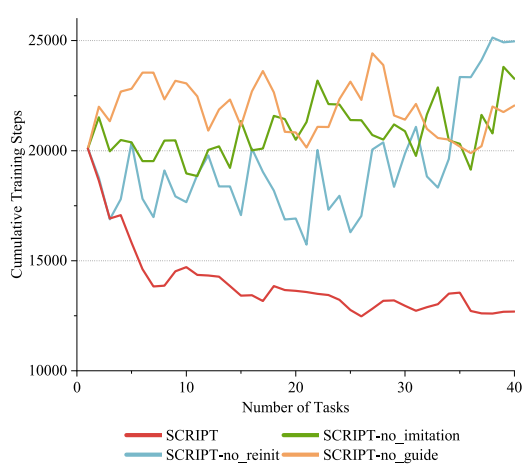

Fig. 8. Cumulative training steps per task as the number of tasks increases during the continual learning process.

6. Experimental results

6.1. Resilience against catastrophic forgetting

This part aims to evaluate agents' resilience against catastrophic forgetting by answering RQ1 and RQ2 as we mentioned in Section 1.

6.1.1. RQ1: Comparison to baseline methods

To investigate whether SCRIPT has better resistance to catastrophic forgetting than other methods (RQ1), we compare it with different baselines by evaluating the following metrics: (a) performance retention during sequential training, (b) forgetfulness on the initial task during sequential training, and (c) the number of memorized tasks after learning all tasks.

The baselines are as follows:

- Fine-tune. As a representative baseline method in the literature (Wang et al., 2023), Fine-tune offers a straightforward method for leveraging existing knowledge while adapting to new tasks.
- Progress&Compress (P&C) (Schwarz et al., 2018). Similar to our method, it separates the knowledge learning and knowledge preservation process, and proposes Online EWC to realize weight consolidation.
- Policy Consolidation (PC) (Kaplanis et al., 2019). Inspired by the synaptic model, it utilizes a cascade of hidden networks. This approach enables the system to remember policies at various timescales while also regularizing the current policy based on its historical data.
- MeDQN (Lan et al., 2023). The memory-efficient deep Q-network algorithm (MeDQN) can consolidate knowledge from the target Q-network to the current Q-network to reduce forgetting and maintain high sample efficiency.

Since there is currently no existing research that applies CRL to autonomous pentesting, the baselines are selected from those proposed in the general domain. These baselines are representative CRL methods that require no extra storing of previous data or task-specific parameters, and no clear task boundaries. They share characteristics aligned with the objectives of this paper and meet the requirements for autonomous pentesting.

Fig. 3 shows the changes in performance retention and zero-shot performance on the initial task across different methods as the number of learning tasks increases. Experimental results in Fig. 3(a) show that as the number of learning tasks increases, different methods exhibit varying degrees of forgetting for previously learned tasks. In comparison, SCRIPT demonstrates a smoother forgetting curve, maintaining better performance retention than others. After learning all tasks, SCRIPT can memorize more than 60% previously learned tasks. Moreover, as depicted in Fig. 4, SCRIPT retains an average of more than 25 tasks (38% more than suboptimal method P&C), which is the highest among other methods. Meanwhile, as shown in the results in Fig. 3(b), SCRIPT maintains better performance on the initially learned tasks compared to other methods, indicating that SCRIPT does not completely forget the knowledge from the initial tasks and exhibits strong long-term memory capabilities.

These results suggest that our method better resists catastrophic forgetting (RQ1) and demonstrates greater robustness as task scaling. As expected, without protection against catastrophic forgetting such as Fine-tune, the agent may become too forgetful to adapt to the demands of continual learning. Consequently, whenever the agent encounters a task, even if the task has been learned before, it must start learning from scratch. Previous studies in autonomous pentesting have frequently overlooked this challenge, but our proposed solution effectively alleviates this issue.

6.1.2. RQ2: Ablation study

To answer RQ2 and gain a deeper understanding of how the weights importance constraint and policy retrospection contribute to the agent's resilience against catastrophic forgetting, we conduct an ablation study by comparing SCRIPT to its two variants:

- SCRIPT-no_wic. Removing the regularization term of weights importance constraint during knowledge consolidation.

- SCRIPT-no_retr. Removing the regularization term of policy retrospection during knowledge consolidation.

We begin by evaluating the performance retention and forgetfulness on the initial task during sequential training. As depicted in Figs. 4 and 5, both the weights importance constraint and policy retrospection contribute to the agents' resilience against catastrophic forgetting (RQ2). Whether it is performance retention across all tasks (Fig. 5(a)) or long-term memory of the initial task (Fig. 5(b)), removing either of the two methods results in a decline in the agent's memory capacity. Additionally, as observed from Fig. 5(a), when the number of learning tasks is relatively small (fewer than 15), the performance retention of SCRIPT-no_wic and SCRIPT-no_retr is quite similar. However, as the number of learning tasks increases, the performance of SCRIPT-no_wic declines significantly, indicating that the weights importance constraint contributes more to the agent's task scalability.

To further investigate the impact of weights importance constraint and policy retrospection on the agent's ability to resist catastrophic forgetting, we extract the top 10%, middle 10%, and last 10% of the training tasks after all tasks are learned. The agent's average zero-shot performance, measured by the cumulative reward, is then evaluated on these three task sets. The agent's performance on these task sets reflects its long-term, middle-term, and short-term memory capabilities, respectively.

As shown in Fig. 6, for tasks learned in the early and mid stages, SCRIPT-no_wic performs worse than SCRIPT-no_retr. However, for tasks learned in the later stages, SCRIPT-no_retr performs worse. This somewhat indicates that the weights importance constraint is more effective in protecting the agent's knowledge acquired in the early and mid stages from being forgotten, while policy retrospection is more beneficial for enhancing the agent's short-term memory capabilities.

6.2. Assessing positive forward transfer

This part aims to investigate the forward transferability of SCRIPT by answering RQ3 and RQ4 as we mentioned in Section 1.

6.2.1. RQ3: Forward transferability of SCRIPT

To investigate whether SCRIPT can effectively utilize previously learned task knowledge to accelerate new task learning (RQ3), we assess the agent's learning performance on the same unseen task $T_m \notin \mathcal{T}$ during sequential training in each run.

From Fig. 7 we can see that utilizing previously learned task knowledge is effective in improving learning efficiency compared to learning from scratch. Furthermore, as the number of learned tasks increases, we observe a corresponding improvement in the agent's learning performance, that is, the agent's convergence speed in learning new tasks accelerates significantly. This indicates that the agent trained with SCRIPT effectively leverages prior task knowledge to expedite the learning process for new tasks, resulting in positive forward transfer.

6.2.2. RQ4: Ablation study

To further assess the impact of the scaffolded curriculum guide (including rolling exploration and policy imitation) and model re-initialization on forward transferability (RQ4), we conduct an ablation study comparing SCRIPT with its three variants:

- SCRIPT-no_reinit. When encountering a new task, the agent retains the previous model parameters to learn the new task, instead of re-initializing.
- SCRIPT-no_imitation. We remove the policy imitation and only apply rolling exploration in the curriculum guide.
- SCRIPT-no_guide. The agent learns from scratch, without a scaffolded curriculum guide, including rolling exploration or policy imitation when encountering a new task.

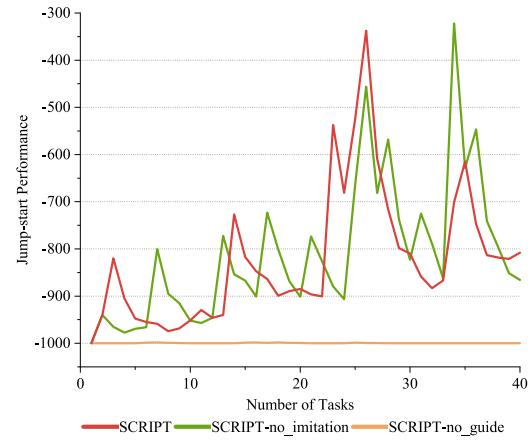

Fig. 9. Jump-start performance of SCRIPT and its two variants.

We begin by comparing the cumulative training steps of SCRIPT and its three variants for each task as the number of training tasks increases.

From Fig. 8 we observe a decreasing trend in SCRIPT's cumulative training steps per task as the number of tasks increases, whereas the other three variants do not show a similar trend. This indicates that both scaffolded curriculum guide and model re-initialization effectively enhance new task learning, and both are indispensable. In addition, from Fig. 8 we can also observe that the cumulative training steps per task for SCRIPT-no_reinit even tend to increase as the number of tasks grows. This phenomenon is known as the catastrophic loss of plasticity, which is described as learning systems exhausting their learning capital over time, ultimately failing to meet the demands of a changing world (Dohare et al., 2021). Besides, the comparison between SCRIPT and SCRIPT-no_guide suggests that the scaffolded curriculum guide plays a key role in achieving positive forward transferability. However, the performance of SCRIPT-no_imitation further indicates that simply using the progressive scaffolding instructional strategy through rolling exploration is insufficient.

To further investigate how rolling exploration and policy imitation in the scaffolded curriculum guide contribute to enhancing the agents' forward transferability, we evaluate two key metrics for SCRIPT, SCRIPT-no_imitation, and SCRIPT-no_guide: jump-start performance and average learning performance across 40 tasks during sequential training.

As is shown in Fig. 9, both SCRIPT and SCRIPT-no_imitation equipped with rolling exploration have better jump-start performance than SCRIPT-no_guide. This suggests rolling exploration can help the agent achieve meaningful rewards in the initial training episode, thereby achieving kickstarting.

Comparing the performance of SCRIPT and SCRIPT-no_imitation in Fig. 10, it can be observed that the learning curve of SCRIPT-no_imitation shows a brief rise followed by a decline. This phenomenon occurs at the end of the scaffolded curriculum guidance. At this point, as described in Section 4.2, due to the presence of distributional shifts, plus the learners' excessive reliance on teacher guidance, they failed to internalize the knowledge, resulting in the scaffolding fading effect. SCRIPT successfully alleviates this phenomenon by employing policy distillation during the curriculum guidance phase, encouraging learners to actively imitate the teacher's policy to facilitate knowledge internalization.

7. Discussion

This section aims to provide deeper reflections on the experimental results, critical discussion of the limitations of our work, and finally offer potential avenues for future improvement.

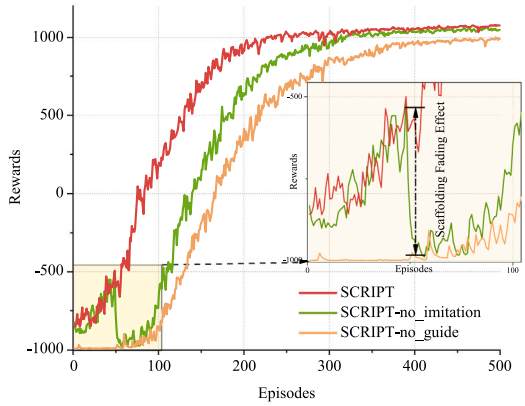

Fig. 10. Average learning performance across 40 sequential training tasks for SCRIPT and its two variants, with the scaffolding fading effect shown in the localized zoomed-in image.

7.1. Reflections on experimental results

7.1.1. Results on resilience against catastrophic forgetting

From the experimental results in Section 6.1, we can observe that the proposed framework SCRIPT outperforms the baselines in mitigating the catastrophic forgetting problem. SCRIPT demonstrates a better forgetting curve during the continual learning process, retaining over 60% of the tasks even after learning 40 tasks, although there is still significant room for improvement. Furthermore, for two techniques used in the proposed framework: weights importance constraint and policy retrospection, we conducted a deeper analysis of their impact on the agent's memory capabilities.

The results in this part show that introducing appropriate regularization terms to constrain the range of model parameter changes effectively improves the agent's memory. Additionally, different regularization terms have varying effects on the agent's short-term and long-term memory. Future work could explore these effects in greater detail, possibly drawing inspiration from research on the human Ebbinghaus Forgetting Curve (Murre & Dros, 2015).

7.1.2. Results on agent's forward transferability

Regarding the agent's forward transferability in Section 6.2, SCRIPT accelerates the learning of new tasks by leveraging past knowledge. The reason for achieving this effect lies in our use of curriculum learning, which guides the learning process of new tasks and shortens the exploration of common knowledge between different tasks. This improvement in the exploration ability can be reflected in the agent's jump-start performance. Additionally, we experimentally validated the role of model re-initialization in addressing the catastrophic loss of plasticity and demonstrated how policy imitation can help mitigate the scaffolding fading effect.

However, we found that the improvement in the agent's forward transferability diminishes as the number of tasks increases (see Fig. 8). This could be due to a limit in the capacity of the teacher policy used to guide new task learning. This capacity limit is likely related to the amount of shared knowledge between tasks. For instance, in the domain of pentesting, this shared knowledge may only extend to the general scanning and probing actions during the early pentesting stages. In contrast, how to infer potential vulnerabilities based on the detected information may not be common knowledge across tasks, and thus the agent's exploration ability cannot be enhanced through the teacher's guidance. Future work could explore combining the SCRIPT with action embedding methods (Zhou, Liu, Lu, Yang, Hou, et al., 2024) or meta-learning to further improve the agent's policy transfer ability.

7.2. Limitations

In this work, we provide a feasible solution to train pentesting agents in a continual fashion and evaluate our method on relatively simple pentesting tasks. However, there still exists a long way to go before such pentesting agents can be actually applied to complex pentesting tasks in real-world settings. On one hand, the vulnerable environments used in our experiments cannot fully replicate the complexity of real-world penetration testing scenarios. On the other hand, the agent's state and action spaces are not comprehensive enough to address the complexities of real-world situations. For instance, when testing real-world hosts, challenges such as security measures (e.g., firewalls, intrusion detection systems), network connectivity issues, or the need for multi-stage exploitation may arise. In the future, we plan to gradually introduce more complex training environments and enhance the completeness of the agent's state and action spaces to improve its applicability in real-world scenarios.

Next, in this work, the design of the reward function relies on subjective estimates from pentesting experts. However, assessing reward function presents a challenge since there is no objective standard defining an "optimal" strategy (Holm, 2023). One potential avenue for future research is to explore automated methods for generating reward functions.

Moreover, how to better represent the state information of RL agents remains an open question. In this paper, we finetune the existing pretrained embedding model as the encoder to better capture the semantic meaning of state information. However, the effectiveness of this approach requires further in-depth evaluation. Evidence has shown that a good state representation method can improve RL agents' learning efficiency and robustness (Raffin et al., 2019). We plan to explore this topic in future research.

Finally, in this work, we primarily validate the proposed framework through quantitative performance evaluations, where we assess its effectiveness via comparison experiments and ablation studies. Future work may include seeking expert feedback to further refine and validate the framework's real-world applicability.

8. Conclusion

In this paper, we present SCRIPT, the first task-scalable CRL framework for autonomous pentesting. We aim to train pentesting agents to learn task sequences continually like humans, while avoiding catastrophic forgetting and being able to leverage previously learned knowledge to accelerate new task learning, so as to better cope with expanded attack surfaces in the real world. The experimental results demonstrate that SCRIPT effectively enables agents to achieve positive forward transfer and enhance their resistance to catastrophic forgetting, making it a feasible approach for pentesting agents to handle the real-world non-stationarity challenge and tackle expanded attack surfaces. By introducing SCRIPT, we not only contribute to the field of autonomous pentesting but also provide a novel CRL framework that can be used in other RL-based applications.

CRediT authorship contribution statement

Shicheng Zhou: Conceptualization, Investigation, Methodology, Software, Validation, Writing – original draft. **Jingju Liu:** Conceptualization, Project administration, Writing – review & editing, Resources. **Yuliang Lu:** Supervision, Project administration, Writing – review & editing. **Jiahai Yang:** Supervision, Writing – review & editing. **Yue Zhang:** Investigation, Visualization, Software. **Bo Lin:** Writing – review & editing. **Xiaofeng Zhong:** Writing – review & editing. **Shulong Hu:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

Abbas, Z., Zhao, R., Modayil, J., White, A., & Machado, M. C. (2023). Loss of plasticity in continual deep reinforcement learning. In *Proceedings of machine learning research: vol. 232, Conference on lifelong learning agents, 22-25 August 2023, mcGill university, montréal, québec, Canada* (pp. 620–636).

Abel, D., Barreto, A., Roy, B. V., Precup, D., van Hasselt, H. P., & Singh, S. (2023). A definition of continual reinforcement learning. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, & S. Levine (Eds.), *Advances in neural information processing systems 36: annual conference on neural information processing systems 2023, neurIPS 2023, new orleans, la, USA, December 10 - 16, 2023* (pp. 1–31).

Aljundi, R., Babiloni, F., Elhoseiny, M., Rohrbach, M., & Tuytelaars, T. (2018). Memory aware synapses: Learning what (not) to forget. In V. Ferrari, M. Hebert, C. Sminchisescu, & Y. Weiss (Eds.), *Lecture notes in computer science: vol. 11207, Computer vision - ECCV 2018 - 15th European conference, munich, Germany, September 8-14, 2018, proceedings, part III* (pp. 144–161). http://dx.doi.org/10.1007/978-3-030-01219-9_9.

Atkinson, C., McCane, B., Szymanski, L., & Robins, A. V. (2021). Pseudo-rehearsal: Achieving deep reinforcement learning without catastrophic forgetting. *Neurocomputing*, *428*, 291–307. http://dx.doi.org/10.1016/J.NEUCOM.2020.11.050.

Brehmer, B. (2005). The dynamic OODA loop: Amalgamating boyd's OODA loop and the cybernetic approach to command and control. *Proceedings International Command &Control Research & Technology Symposium*.

Chen, J., Hu, S., Zheng, H., Xing, C., & Zhang, G. (2023). GAIL-PT: An intelligent penetration testing framework with generative adversarial imitation learning. *Computers & Security*, *126*, Article 103055.

Czarnecki, W. M., Pascanu, R., Osindero, S., Jayakumar, S., Swirszcz, G., & Jaderberg, M. (2019). Distilling policy distillation. In K. Chaudhuri, & M. Sugiyama (Eds.), *Proceedings of machine learning research: vol. 89, Proceedings of the twenty-second international conference on artificial intelligence and statistics* (pp. 1331–1340). PMLR.

Dohare, S., Mahmood, A. R., & Sutton, R. S. (2021). Continual backprop: Stochastic gradient descent with persistent randomness. *CoRR*, arXiv:2108.06325.

Feng, S., Sun, H., Yan, X., Zhu, H., Zou, Z., Shen, S., & Liu, H. X. (2023). Dense reinforcement learning for safety validation of autonomous vehicles. *Nature*, *615*, 620–627.

François-Lavet, V., Henderson, P., Islam, R., Bellemare, M. G., & Pineau, J. (2018). *An Introduction to Deep Reinforcement Learning*. Now Foundations and Trends, http://dx.doi.org/10.1561/2200000071.

Gao, C., Gao, H., Guo, S., Zhang, T., & Chen, F. (2021). CRIL: continual robot imitation learning via generative and prediction model. In *IEEE/RSJ international conference on intelligent robots and systems, IROS 2021, prague, czech Republic, September 27 - oct. 1, 2021*. http://dx.doi.org/10.1109/IROS51168.2021.9636069.

Gaya, J., Doan, T., Caccia, L., Soulier, L., Denoyer, L., & Raileanu, R. (2023). Building a subspace of policies for scalable continual learning. In *The eleventh international conference on learning representations, ICLR 2023, kigali, rwanda, May 1-5, 2023* (pp. 1–28). OpenReview.net.

Holm, H. (2023). Lore a red team emulation tool. *IEEE Trans. Dependable Secur. Comput.*, *20*(2), 1596–1608. http://dx.doi.org/10.1109/TDSC.2022.3160792.

Hore, S., Shah, A., & Bastian, N. D. (2023). Deep VULMAN: a deep reinforcement learning-enabled cyber vulnerability management framework. *Expert Systems with Applications*, *221*, Article 119734. http://dx.doi.org/10.1016/J.ESWA.2023.119734.

Hung, S. C. Y., Tu, C., Wu, C., Chen, C., Chan, Y., & Chen, C. (2019). Compacting, picking and growing for unforgetting continual learning. In *Advances in neural information processing systems 32: annual conference on neural information processing systems 2019, neurIPS 2019, December 8-14, 2019, vancouver, BC, Canada* (pp. 13647–13657).

Igl, M., Farquhar, G., Luketina, J., Boehmer, W., & Whiteson, S. (2021). Transient non-stationarity and generalisation in deep reinforcement learning. In *9th international conference on learning representations, ICLR 2021, virtual event, Austria, May 3-7, 2021* (pp. 1–16).

Ilic, N., Dasic, D., Vucetic, M., Makarov, A., & Petrovic, R. (2024). Distributed web hacking by adaptive consensus-based reinforcement learning. *Artificial Intelligence*, *326*, Article 104032. http://dx.doi.org/10.1016/J.ARTINT.2023.104032.

Kaplanis, C., Shanahan, M., & Clopath, C. (2019). Policy consolidation for continual reinforcement learning. In *Proceedings of machine learning research: vol. 97, Proceedings of the 36th international conference on machine learning, ICML 2019, 9-15 June 2019, long beach, california, USA* (pp. 3242–3251).

Khetarpal, K., Riemer, M., Rish, I., & Precup, D. (2022). Towards continual reinforcement learning: A review and perspectives. *Journal of Artificial Intelligence Research*, *75*, 1401–1476. http://dx.doi.org/10.1613/JAIR.1.13673.

Kim, K., Ji, B., Yoon, D., & Hwang, S. (2020). Self-knowledge distillation with progressive refinement of targets. *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, 6547–6556. http://dx.doi.org/10.1109/ICCV48922.2021.00650.

Kirkpatrick, J., Pascanu, R., Rabinowitz, N. C., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., Hassabis, D., Clopath, C., Kumaran, D., & Hadsell, R. (2016). Overcoming catastrophic forgetting in neural networks. *CoRR*, arXiv:1612.00796.

Lan, Q., Pan, Y., Luo, J., & Mahmood, A. R. (2023). Memory-efficient reinforcement learning with value-based knowledge consolidation. *Trans. Mach. Learn. Res.*, *2023*.

Li, Q., Wang, R., Li, D., Shi, F., Zhang, M., Chattopadhyay, A., Shen, Y., & Li, Y. (2024). DynPen: Automated penetration testing in dynamic network scenarios using deep reinforcement learning. *IEEE Transactions on Information Forensics and Security*, *19*, 8966–8981. http://dx.doi.org/10.1109/TIFS.2024.3461950.

Li, Q., Zhang, M., Shen, Y., Wang, R., Hu, M., Li, Y., & Hao, H. (2023). A hierarchical deep reinforcement learning model with expert prior knowledge for intelligent penetration testing. *Computers & Security*, *132*, Article 103358. http://dx.doi.org/10.1016/J.COSE.2023.103358.

Maeda, R., & Mimura, M. (2021). Automating post-exploitation with deep reinforcement learning. *Computers & Security*, *100*, Article 102108.

Microsoft Defender Research Team (2021). Cyberbattlesim. Created by Christian Seifert, Michael Betser, William Blum, James Bono, Kate Farris, Emily Goren, Justin Grana, Kristian Holsheimer, Brandon Marken, Joshua Neil, Nicole Nichols, Jugal Parikh, Haoran Wei. https://github.com/microsoft/cyberbattlesim,

Murre, J., & Dros, J. (2015). Replication and analysis of ebbinghaus' forgetting curve. *PloS One*, *10*, Article e0120644. http://dx.doi.org/10.1371/journal.pone.0120644.

Nguyen, H. P. T., Chen, Z., Hasegawa, K., Fukushima, K., & Beuran, R. (2024). PenGym: Pentesting training framework for reinforcement learning agents. In G. Lenzini, P. Mori, & S. Furnell (Eds.), *Proceedings of the 10th international conference on information systems security and privacy, ICISSP 2024, rome, Italy, February 26-28, 2024* (pp. 498–509). http://dx.doi.org/10.5220/0012367300003648.

O'Rourke, E., Andersen, E., Gulwani, S., & Popovic, Z. (2015). A framework for automatically generating interactive instructional scaffolding. In B. Begole, J. Kim, K. Inkpen, & W. Woo (Eds.), *Proceedings of the 33rd annual ACM conference on human factors in computing systems, CHI 2015, seoul, Republic of Korea, April 18-23, 2015* (pp. 1545–1554). http://dx.doi.org/10.1145/2702123.2702580.

Parisi, G. I., Kemker, R., Part, J. L., Kanan, C., & Wermter, S. (2019). Continual lifelong learning with neural networks: A review. *Neural Networks*, *113*, 54–71. http://dx.doi.org/10.1016/J.NEUNET.2019.01.012.

Powers, S., Xing, E., Kolve, E., Mottaghi, R., & Gupta, A. (2022). CORA: benchmarks, baselines, and metrics as a platform for continual reinforcement learning agents. In S. Chandar, R. Pascanu, & D. Precup (Eds.), *Proceedings of machine learning research: vol. 199, Conference on lifelong learning agents, coLLAs 2022, 22-24 August 2022, mcGill university, montréal, québec, Canada* (pp. 705–743).

Raffin, A., Hill, A., Traoré, K. R., Lesort, T., Rodríguez, N. D., & Filliat, D. (2019). Decoupling feature extraction from policy learning: assessing benefits of state representation learning in goal based robotics. *CoRR*, arXiv:1901.08651.

Riemer, M., Cases, I., Ajemian, R., Liu, M., Rish, I., Tu, Y., & Tesauro, G. (2019). Learning to learn without forgetting by maximizing transfer and minimizing interference. In *7th international conference on learning representations, ICLR 2019, new orleans, la, USA, May 6-9, 2019* (pp. 1–31). OpenReview.net.

Ring, M. B. (1995). *Continual learning in reinforcement environments* (Ph.D. thesis), University of Texas at Austin, TX, USA.

Rolnick, D., Ahuja, A., Schwarz, J., Lillicrap, T. P., & Wayne, G. (2019). Experience replay for continual learning. In *Advances in neural information processing systems 32: annual conference on neural information processing systems 2019, neurIPS 2019, December 8-14, 2019, vancouver, BC, Canada* (pp. 348–358).

Rusu, A. A., Colmenarejo, S. G., Gülçehre, Ç., Desjardins, G., Kirkpatrick, J., Pascanu, R., Mnih, V., Kavukcuoglu, K., & Hadsell, R. (2016). Policy distillation. In Y. Bengio, & Y. LeCun (Eds.), *4th international conference on learning representations, ICLR 2016, san juan, puerto rico, May 2-4, 2016, conference track proceedings* (pp. 1–13).

Rusu, A. A., Rabinowitz, N. C., Desjardins, G., Soyer, H., Kirkpatrick, J., Kavukcuoglu, K., Pascanu, R., & Hadsell, R. (2016). Progressive neural networks. *CoRR*, arXiv:1606.04671.

Schneier, B. (1999). Attack trees. *Dr. Dobb's Journal*, *24*(12), 21–29.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. *CoRR*, arXiv:1707.06347.

Schwartz, J., & Kurniawati, H. (2019). Autonomous penetration testing using reinforcement learning. arXiv preprint arXiv:1905.05965.

Schwartz, J., & Kurniawatti, H. (2019). NASim: Network attack simulator. https://github.com/Jjschwartz/NetworkAttackSimulator,

Schwarz, J., Czarnecki, W., Luketina, J., Grabska-Barwinska, A., Teh, Y. W., Pascanu, R., & Hadsell, R. (2018). Progress & compress: A scalable framework for continual learning. In *Proceedings of machine learning research: vol. 80, Proceedings of the 35th international conference on machine learning, ICML 2018, stockholmsmässan, stockholm, Sweden, July 10-15, 2018* (pp. 4535–4544).

Sheyner, O., Haines, J. W., Jha, S., Lippmann, R., & Wing, J. M. (2002). Automated generation and analysis of attack graphs. *Proceedings 2002 IEEE Symposium on Security and Privacy*, 273–284.

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction*. MIT Press.

Takaesu, I. (2018). Deepexploit. https://github.com/13o-bbr-bbq/machine_learning_security/blob/master/DeepExploit.

Tran, K., Akella, A., Standen, M., Kim, J., Bowman, D., Richer, T., & Lin, C.-T. (2021). Deep hierarchical reinforcement agents for automated penetration testing. http://dx.doi.org/10.48550/arXiv.2109.06449, ArXiv E-Prints arXiv:2109.06449.

Uchendu, I., Xiao, T., Lu, Y., Zhu, B., Yan, M., Simon, J., Bennice, M., Fu, C., Ma, C., Jiao, J., Levine, S., & Hausman, K. (2023). Jump-start reinforcement learning. In A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, & J. Scarlett (Eds.), *Proceedings of machine learning research: 202, International conference on machine learning, ICML 2023, 23-29 July 2023, honolulu, hawaii, USA* (pp. 34556–34583).

Van de Pol, J., Volman, M., & Beishuizen, J. (2010). Scaffolding in teacher–student interaction: A decade of research. *Educational Psychology Review*, *22*, 271–296. http://dx.doi.org/10.1007/s10648-010-9127-6.

Vinyals, O., Babuschkin, I., Czarnecki, W. M., Mathieu, M., Dudzik, A., Chung, J., Choi, D. H., Powell, R., Ewalds, T., Georgiev, P., Oh, J., Horgan, D., Kroiss, M., Danihelka, I., Huang, A., Sifre, L., Cai, T., Agapiou, J. P., Jaderberg, M., ..., Vezhnevets, A. S. et al. (2019). Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 1–5.

Wang, Z., Chen, C., & Dong, D. (2022). Lifelong incremental reinforcement learning with online Bayesian inference. *IEEE Trans. Neural Networks Learn. Syst.*, *33*(8), 4003–4016. http://dx.doi.org/10.1109/TNNLS.2021.3055499.

Wang, Z., Chen, C., & Dong, D. (2023). A Dirichlet process mixture of robust task models for scalable lifelong reinforcement learning. *IEEE Trans. Cybern.*, *53*(12), 7509–7520. http://dx.doi.org/10.1109/TCYB.2022.3170485.

Wang, K., Reimers, N., & Gurevych, I. (2021). TSDAE: Using transformer-based sequential denoising auto-encoderfor unsupervised sentence embedding learning. arXiv preprint arXiv:2104.06979.

Wang, L., & Yoon, K. (2022). Knowledge distillation and student-teacher learning for visual intelligence: A review and new outlooks. *IEEE Trans. Pattern Anal. Mach. Intell.*, *44*(6), 3048–3068. http://dx.doi.org/10.1109/TPAMI.2021.3055564.

Wang, L., Zhang, X., Su, H., & Zhu, J. (2024). A comprehensive survey of continual learning: Theory, method and application. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *PP*, http://dx.doi.org/10.1109/tpami.2024.3367329.

Yang, Y., Chen, M., Fu, H., & Liu, X. (2023). SetTron: Towards better generalisation in penetration testing with reinforcement learning. In *IEEE global communications conference, GLOBECOM 2023, kuala lumpur, Malaysia, December 4-8, 2023* (pp. 4662–4667). http://dx.doi.org/10.1109/GLOBECOM54140.2023.10437804.

Yang, Y., Chen, L., Liu, S., Wang, L., Fu, H., Liu, X., & Chen, Z. (2025). Behaviour-diverse automatic penetration testing: a coverage-based deep reinforcement learning approach. *Frontiers Comput. Sci.*, *19*(3), Article 193309. http://dx.doi.org/10.1007/S11704-024-3380-1.

Yoon, J., Yang, E., Lee, J., & Hwang, S. J. (2018). Lifelong learning with dynamically expandable networks. In *6th international conference on learning representations, ICLR 2018, vancouver, BC, Canada, April 30 - May 3, 2018, conference track proceedings* (pp. 1–11). OpenReview.net.

Zenke, F., Poole, B., & Ganguli, S. (2017). Continual learning through synaptic intelligence. In D. Precup, & Y. W. Teh (Eds.), *Proceedings of machine learning research: 70, Proceedings of the 34th international conference on machine learning, ICML 2017, sydney, NSW, Australia, 6-11 August 2017* (pp. 3987–3995).

Zhang, T., Lin, Z., Wang, Y., Ye, D., Fu, Q., Yang, W., Wang, X., Liang, B., Yuan, B., & Li, X. (2023). Dynamics-adaptive continual reinforcement learning via progressive contextualization. *IEEE Transactions on Neural Networks and Learning Systems*, 1–15. http://dx.doi.org/10.1109/TNNLS.2023.3280085.

Zhou, F., & Cao, C. (2021). Overcoming catastrophic forgetting in graph neural networks with experience replay. In *Thirty-fifth AAAI conference on artificial intelligence, AAAI 2021, thirty-third conference on innovative applications of artificial intelligence, IAAI 2021, the eleventh symposium on educational advances in artificial intelligence, EAAI 2021, virtual event, February 2-9, 2021* (pp. 4714–4722). http://dx.doi.org/10.1609/AAAI.V35I5.16602.

Zhou, S., Liu, J., Lu, Y., Yang, J., Hou, D., Zhang, Y., & Hu, S. (2024). APRIL: towards scalable and transferable autonomous penetration testing in large action space via action embedding. *IEEE Transactions on Dependable and Secure Computing*, 1–17. http://dx.doi.org/10.1109/TDSC.2024.3518500.

Zhou, S., Liu, J., Lu, Y., Yang, J., Zhang, Y., & Chen, J. (2024). Towards generalizable autonomous penetration testing via domain randomization and meta-reinforcement learning. http://dx.doi.org/10.48550/arXiv.2412.04078, ArXiv E-Prints arXiv:2412.04078.

Zhu, Z., Lin, K., Jain, A. K., & Zhou, J. (2023). Transfer learning in deep reinforcement learning: A survey. *IEEE Trans. Pattern Anal. Mach. Intell.*, *45*(11), 13344–13362. http://dx.doi.org/10.1109/TPAMI.2023.3292075.